

Module 3 Lab: Malware Threats — Worms, Viruses, Trojans

Password Cracking, Dictionary Attacks, Encryption, DoS, ARP Poisoning, Windows Networking Commands, and Steganography

Course: Ethical Hacking Lab — Module 3 (matches your uploaded manual's practical sequence, with every syllabus item built out as a full task) **Estimated time:** 5–6 hours — can span two sessions **Format:** Individual; uses your isolated host-only lab plus one Windows VM **Prerequisite:** Labs 1–4. A `baseline-pre-attack` snapshot of Metasploitable 2 available.

A Note on Tool Choice

Your manual's practicals for this module are built around **Cain & Abel**: a Windows hash calculator and dictionary-attack tool, and separately its ARP poisoning ("APR") and password-sniffing feature. One substitution from the manual: Cain & Abel's official site has been offline for years and the tool hasn't been updated since around 2014, so the only way to get a copy today is from an unverified third-party mirror — not something worth the risk for an executable with this much system access, when the underlying lesson doesn't actually require that specific tool.

This lab teaches the identical sequence using tools that are current, cross-platform, and verifiable: **CrypTool** (exactly as your manual specifies) for encryption and hashing, **Hashcat** for the dictionary attack, and your existing Kali setup for the ARP poisoning — which also happens to be how this attack runs in the real world, from an attacker-controlled Linux box against a Windows victim, not from a GUI tool installed on Windows itself.

1. Learning Objectives

By the end of this session you will be able to:

- Encrypt and decrypt a password using a modern symmetric cipher, and explain what the key actually protects
- Generate a password hash and recover the original password from it using a dictionary attack
- Use core Windows networking commands to read a machine's configuration and connectivity
- Poison a Windows target's ARP cache from Kali, observe the poisoned entry yourself, and capture credentials crossing the resulting path
- Launch and observe a denial-of-service condition inside a fully isolated environment, and name a real defense against it
- Hide a message inside an image with steganography, recover it, and break a weak steganography passphrase
- Identify and briefly evaluate at least two additional security tools outside this lab's scripted tasks

2. ⚠ Scope Reminder

Every technique below — including the denial-of-service and ARP poisoning tasks — runs entirely inside your own isolated host-only network, against VMs you built yourself. These two techniques in particular are not ones you'd ever run against a network you don't personally own and control: they don't just reveal information, they actively disrupt or redirect traffic for every device on the segment they touch.

3. Environment Note

This lab needs a Windows VM on the same host-only network as Kali and Metasploitable 2.

1. Download a free, official Windows evaluation VM from Microsoft's developer site (legitimate, time-limited VirtualBox images meant exactly for testing like this).
2. Import it the way you imported Kali and Metasploitable back in Lab 1: **Settings → Network → Adapter 1 → Host-only Adapter**, the same adapter your other VMs use, with **Adapters 2–4 disabled**.
3. Boot it, confirm it picks up an address on your `192.168.56.0/24` range, and take a `baseline-pre-attack` snapshot before you touch anything else in this lab.

4. What You'll Need

- Kali, Metasploitable 2, and the Windows VM, all running and mutually reachable
- CrypTool (free, actively maintained — install on the Windows VM, or on Kali via Wine)
- Hashcat (pre-installed on Kali; also has an official Windows build)
- Wireshark (pre-installed on Kali)
- `arpspoof` (Kali; `sudo apt install dsniiff` if not already present)
- `hping3`, `steghide`, `stegseek` (all pre-installed on Kali, or `sudo apt install` if not)

5. Procedure

Part A — Encrypting and Decrypting a Password with CrypTool

1. Launch CrypTool and open the **Encrypt/Decrypt** menu.
2. Select **Symmetric (Modern)**, then choose the **RC4** algorithm.
3. Enter a sample password as your plaintext (e.g., `MySecret123`) and a key of your choosing, then run the encryption. Record the resulting ciphertext.
4. Reverse the process: feed your ciphertext and the same key back through CrypTool's decryption function, and confirm you recover the original password exactly.
5. **Reflection (1-2 sentences):** What specifically would an attacker need to reverse this — the ciphertext alone, or the ciphertext and the key? What does that tell you about where the real protection in encryption actually lives?

Part B — Password Cracking: Hashing and a Dictionary Attack

1. In CrypTool, find the **hash** function (**Digest** or **Hash** menu, depending on version) and compute the **MD5** hash of a password from a common wordlist — for example, `dragon`. Record the resulting hash value.

2. Save that hash into a text file on Kali:

```
echo "<paste the MD5 hash here>" > target_hash.txt
```

3. Run Hashcat's dictionary attack against it:

```
hashcat -m 0 -a 0 target_hash.txt /usr/share/wordlists/rockyou.txt
```

4. Confirm Hashcat recovers `dragon` and record how long it took.
5. Repeat Steps 1–4 with a second, stronger password of your choosing (something not likely to appear in a common wordlist) and record whether it cracked at all within a reasonable wait.
6. **Reflection (2-3 sentences):** You generated a hash the same way you generated ciphertext in Part A — both started as a password and ended as a string that doesn't look like the original. Why did Part A's result come back with a key, while this one only came back through guessing?

Part C — Core Windows Networking Commands

1. On the Windows VM, open Command Prompt and run each of the following, recording the output:

```
ipconfig /all  
ping 192.168.56.1  
netstat -ano  
tracert www.google.com  
arp -a
```

2. For each command, write one sentence on what it actually told you — `ipconfig` shows this machine's own configuration; `ping` confirms reachability; `netstat` shows active connections; `tracert` shows the path to a destination; `arp -a` shows which MAC address this machine currently believes belongs to which IP address.
3. Keep this terminal open — you'll run `arp -a` again in Part D to see it change in real time.

Part D — ARP Poisoning the Windows Target

1. On Kali, confirm the Windows VM's IP address and the host-only gateway address (`192.168.56.1`).
2. Start a Wireshark capture on Kali's host-only interface.
3. Launch the ARP poisoning attack from Kali, redirecting the Windows VM's traffic to the gateway through Kali first:

```
sudo arpspoof -i eth0 -t <windows-vm-ip> 192.168.56.1
```

Leave this running.

4. Back on the Windows VM, run `arp -a` again (from Part C) and compare the gateway's MAC address against what you recorded before — it should now show Kali's MAC address instead of the real gateway's.
5. From the Windows VM, log into an unencrypted test service on your lab network — Metasploitable's FTP service works well:

```
ftp 192.168.56.102
```

(use the `msfadmin`/`msfadmin` credentials from earlier labs)

- Back on Kali, stop the Wireshark capture and filter on `ftp`. Find the `PASS` command and confirm you can read the credential in plain text — exactly what Cain & Abel's password-sniffing feature would have shown you, captured instead through the poisoned path you just built.
- Stop the ARP spoofing (Ctrl+C) and confirm `arp -a` on the Windows VM eventually returns to the correct gateway address.

Part E — Denial-of-Service Attack

- Open a second terminal and SSH into Metasploitable so you can watch its own resource usage during the test:

```
ssh msfadmin@192.168.56.102
top
```

- From Kali, launch a brief SYN flood against Metasploitable's web service:

```
sudo hping3 -S --flood -V -p 80 192.168.56.102
```

- Let this run for no more than 10-15 seconds**, watching the `top` output on Metasploitable for a spike in load or connection handling, then stop it with Ctrl+C. There is no reason to run this longer — the point is observing the effect, not stress-testing your own VM into instability.
- Record what you observed on the target side during the flood — connection counts, CPU/load spikes, or any delay in responsiveness.
- Try the same brief test against the Windows VM's file-sharing port (445) instead, and record whether the effect looked different between the two targets.
- Research and name **one** real defense against this specific technique (SYN cookies, rate limiting, or a dedicated DoS-mitigation appliance/service) and write one sentence on how it works.

Part F — Steganography: Hiding and Finding Data

- Hide a short secret message inside an image using `steghide`:

```
echo "The gateway MAC was spoofed at 10:42" > secret.txt
steghide embed -cf any_image.jpg -ef secret.txt -p "hiddenpass123"
```

- Confirm the image still opens normally and looks unchanged, and record the file size before and after embedding.
- Extract your own message back out, to confirm the round trip works:

```
steghide extract -sf any_image.jpg -p "hiddenpass123"
```

- Now play attacker: re-embed your message using a **weaker** passphrase (like `password1`), then attack your own file blind with `stegseek`'s wordlist search:

```
stegseek any_image.jpg /usr/share/wordlists/rockyou.txt
```

- Record whether it recovered your passphrase and the hidden message.
- Reflection (1-2 sentences):** Part B cracked a weak password hash in seconds. Why does the exact same weakness — a guessable passphrase — undermine steganography here just as badly as it undermined the password in Part B?

6. Self-Learning Topics: Using Additional Hacking Tools

Pick **two** tools from the table below that weren't used anywhere else in this lab. Install each, read its `--help` output or a quickstart guide, and try one basic, non-destructive action against your own lab targets. This section is intentionally not scripted step-by-step — the skill being practiced is reading documentation and getting a new tool working on your own, which is most of what real tool adoption looks like on the job.

--	--	--

Tool	Category	What it's generally used for
Medusa	Password attacks	A Hydra alternative with different protocol support and performance characteristics
Responder	Network poisoning	Captures credentials via LLMNR/NBT-NS spoofing on a Windows-heavy network
CrackMapExec	Post-exploitation/enumeration	Sweeps a network for valid credentials and misconfigurations across many hosts at once
BeEF	Web/browser exploitation	Hooks a browser to demonstrate what's reachable once a user visits a malicious page
OWASP ZAP	Web app testing	An open-source alternative to Burp Suite, useful for automated baseline scans
netcat (<code>nc</code>)	General networking	The "Swiss Army knife" for banner grabbing, simple file transfer, and ad-hoc listeners

For each of your two tools, record: what you ran, what happened, and one limitation or surprise you ran into that the documentation didn't warn you about.

7. Unit End Exercise

Crack a password using a dictionary attack, start to finish, and write up the full chain — the password you chose, its hash, the wordlist and tool you used, the time it took, and (from Part B's second attempt) one password that didn't crack. Explain in 2-3 sentences what made the difference.

8. What to Submit

1. Part A — your encryption/decryption results and reflection
2. Part B — your two hash-cracking attempts (one successful, one not) and reflection
3. Part C — your five command outputs and one-sentence explanations
4. Part D — your before/after `arp -a` output and the captured FTP credential (redact the actual password value in your writeup)
5. Part E — your observed DoS impact against both targets and named defense
6. Part F — your steghide round-trip, stegseek attack result, and reflection
7. Section 6 — your two self-learning tool write-ups
8. Section 7 — the unit end exercise write-up

9. Troubleshooting

Symptom	Likely cause / fix
CrypTool's hash menu looks different from these steps	Menu layout varies by CrypTool version (CT1 vs. CT2) — look for "Hash" or "Digest" under the same general Encrypt/Decrypt or Individ. Procedures area
Hashcat reports no devices found	Kali in a VM often has no GPU passthrough — add <code>--force</code> to run CPU-only; slower, but fine for this lab
Windows VM's <code>arp -a</code> never changes in Part D	Confirm <code>arp spoof</code> 's target IP matches the Windows VM exactly, and both VMs share the same host-only adapter
No <code>PASS</code> packet in the Wireshark filter	Confirm the capture was running on the host-only interface, and that the FTP login happened while both the capture and the spoofing were active
<code>hping3 --flood</code> seems to do nothing visible	Confirm you're watching <code>top</code> on the target, not on Kali — the effect shows up on the receiving end
<code>steghide</code> / <code>stegseek</code> not found	<code>sudo apt install steghide stegseek</code>

This is a Module 3 companion lab, built to match your uploaded manual's practical sequence while giving every syllabus item — including DoS and steganography — a full hands-on task rather than a passing mention. If you'd like the later modules from the same manual (web server/app attacks, SQL injection and session hijacking, wireless and

cryptography, or Metasploit penetration testing) built the same way, just say the word.